

DAO GOVERNANCE ARCHITECTURE

Layer 2: State & Execution Layer

Horizontal Decomposition for Agentic Coding

Derived from the Self-Evolving & Self-Optimizing

Quadratic Voting Module Architecture

Document Date: April 2026

Executive Summary

This document provides a complete horizontal decomposition of Layer 2 (State & Execution Layer) from the DAO Governance Architecture. Layer 2 is the immutable, deterministic core that processes quadratic mathematics, manages Voice Credits, and executes passed proposals.

The decomposition splits Layer 2 into six specialized, independently testable agents. Each agent owns a distinct functional domain with clear interfaces, state ownership, input/output contracts, and mathematical invariants. This structure is designed specifically for agentic coding -- enabling multiple AI coding agents to implement, test, and audit each component in parallel without cross-contamination.

Design Principle: Each agent communicates only through well-defined interfaces. No agent may directly mutate another agent's state. All cross-agent calls are explicit and auditable.

Architecture Overview

Layer 2 Core Mandate

Per the original architecture, Layer 2 must satisfy three absolute constraints:

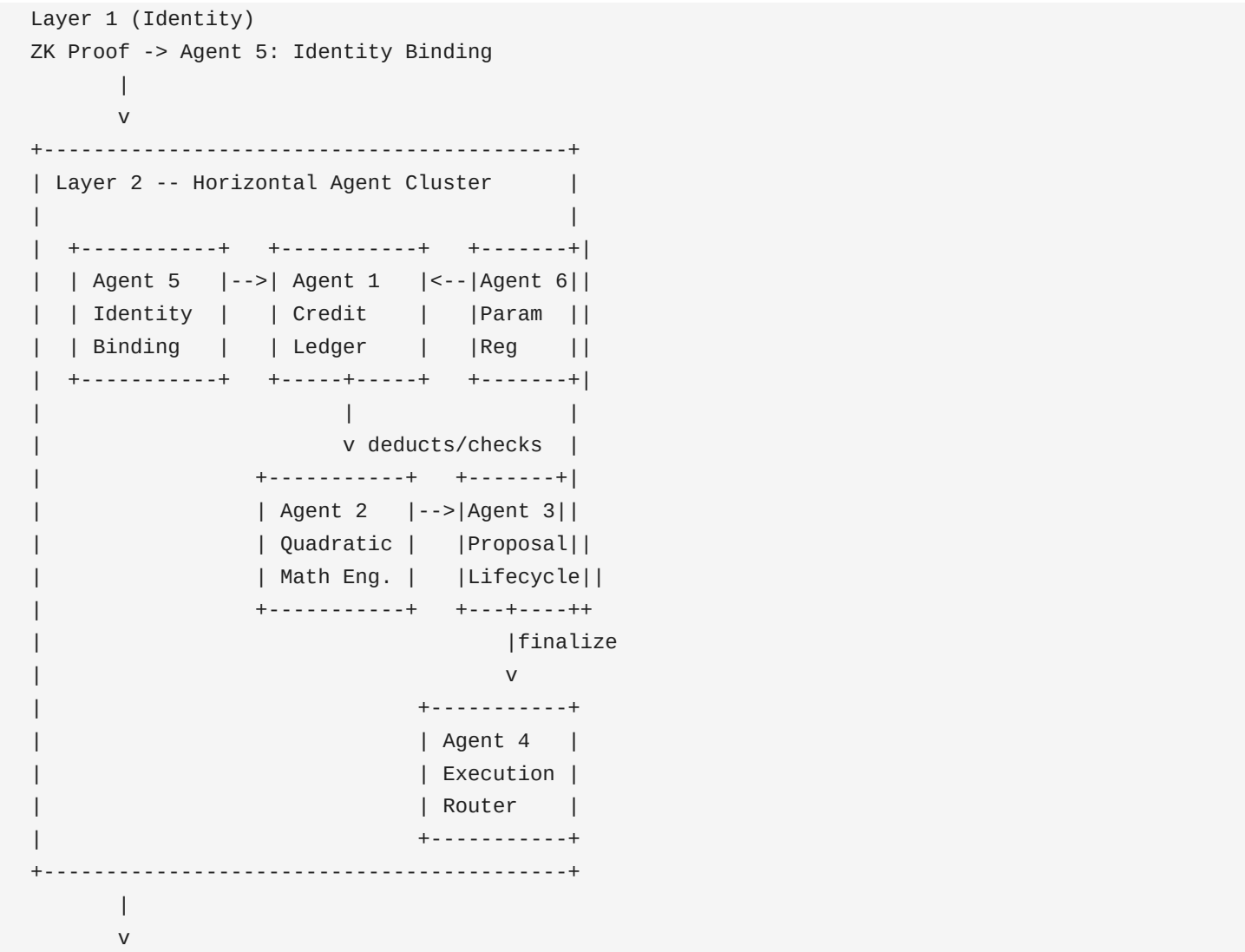
- Core contract logic must be non-upgradable to ensure baseline trust.
- Execution must be purely deterministic with no external dependencies for core state transitions.
- When a user votes, the contract calculates $\text{Cost} = \text{SUM}(\text{vote}_i)^2$ and deducts it from allocated Voice Credits.

Horizontal Decomposition Rationale

A horizontal decomposition separates concerns by functional domain rather than by vertical flow. This ensures that each agent can be developed, tested, and formally verified independently. The six agents are:

- Agent 1: Voice Credit Ledger Agent -- Credit accounting and regeneration
- Agent 2: Quadratic Math Engine Agent -- Pure quadratic voting mathematics
- Agent 3: Proposal Lifecycle Agent -- Proposal state machine and governance rules
- Agent 4: Execution Router Agent -- Deterministic execution of passed proposals
- Agent 5: Identity Binding Agent -- Bridge between Layer 1 and Layer 2
- Agent 6: Parameter Registry Agent -- Governance parameter storage and bounded mutation

Cross-Agent Interaction Flow



DAO Governance Architecture -- Layer 2 Horizontal Decomposition

Layer 3 (Intelligence/Optimization)

Layer 4 (Oracles)

Layer 5 (Veto/Timelock)

Agent 1: Voice Credit Ledger Agent

Domain

Voice Credit accounting, allocation, and regeneration.

Responsibilities

- Maintain the canonical ledger of Voice Credit balances per verified identity
- Handle initial credit allocation (e.g., per-epoch airdrop, staking-derived)
- Process credit deductions when votes are cast (called by Agent 2)
- Manage credit regeneration schedules and rates
- Emit balance change events for Layer 4 (Oracle/Analytics)

State Ownership

```
mapping(identity => uint256) creditBalance
mapping(identity => uint256) lastRegenerationTimestamp
uint256 globalRegenerationRate // read from Agent 6
```

Input Interface

```
allocateCredits(identity, amount) // from Layer 1 post-verification
deductCredits(identity, amount)   // from Agent 2 only
regenerateCredits(identity)       // callable by anyone (idempotent)
```

Output Interface

```
getBalance(identity) -> uint256
canAfford(identity, cost) -> bool
```

Invariants

- Credits can never go negative
- Deductions only succeed if balance $\geq$ cost
- Regeneration is deterministic and time-based only

Agent 2: Quadratic Math Engine Agent

Domain

Pure quadratic voting mathematics and vote aggregation.

Responsibilities

- Calculate vote cost: $\text{Cost} = \text{SUM}(v_i)^2$ for each proposal
- Validate that submitted votes are mathematically legal ($\text{cost} \leq \text{available credits}$)
- Aggregate votes per proposal: running tally of (option, votePower)
- Compute final results: winning option, total participation, quadratic distribution metrics
- Must be purely deterministic -- no external calls, no oracles, no timestamps

State Ownership

```
mapping(proposalId => mapping(optionId => uint256)) voteTally
mapping(proposalId => mapping(identity => Vote)) votesCast
mapping(proposalId => uint256) totalVoiceCreditsSpent
```

Input Interface

```
castVote(proposalId, identity, optionId, votePower, signature)
getCost(votePower) -> uint256    // pure function
```

Output Interface

```
getResults(proposalId) -> (winningOption, totalVotes, distributionMetrics)
getVoteCost(proposalId, identity, newVotePower) -> uint256    // for UI preview
```

Invariants

- $\text{totalVoiceCreditsSpent} = \text{SUM}(\text{individualCosts})$ always holds
- One vote per identity per proposal (overwrites allowed with recalculation)
- No external state reads (block.timestamp, oracle, etc.)

CRITICAL: This agent contains the mathematical heart of the system. Any bug here collapses the entire quadratic voting guarantee. Fuzz testing and formal verification are mandatory.

Agent 3: Proposal Lifecycle Agent

Domain

Proposal state machine and governance rules enforcement.

Responsibilities

- Manage proposal states: Draft -> Active -> Passed/Failed -> Executed/Expired
- Enforce voting duration (read from Agent 6)
- Check quorum and passing thresholds at proposal conclusion
- Coordinate state transitions by calling Agent 2 (for final tally) and Agent 4 (for execution)
- Prevent duplicate or conflicting active proposals where configured

State Ownership

```
mapping(proposalId => Proposal) proposals
mapping(proposalId => ProposalStatus) status
uint256 activeProposalCount
```

Input Interface

```
submitProposal(metadata, executionPayload)    // from Layer 1 verified identities
finalizeProposal(proposalId)                  // callable by anyone after voting period
cancelProposal(proposalId)                    // only if veto triggered (from Agent 5/Layer 5)
```

Output Interface

```
getStatus(proposalId) -> ProposalStatus
isExecutable(proposalId) -> bool
```

Invariants

- A proposal can only move forward in the state machine
- finalizeProposal only succeeds if block.timestamp >= startTime + duration (duration from Agent 6)
- Execution only triggered if quorum met AND majority threshold passed

Agent 4: Execution Router Agent

Domain

Deterministic execution of passed proposals.

Responsibilities

- Receive validated execution payloads from Agent 3
- Route calls to treasury, external protocols, or internal parameter updates
- Maintain an execution queue with nonce-based ordering
- Ensure atomicity: either full execution or full revert
- Cannot call back into Agent 2 or Agent 1 to prevent reentrancy

State Ownership

```
mapping(proposalId => bytes) executionPayloads
mapping(proposalId => bool) executed
uint256 executionNonce
```

Input Interface

```
queueExecution(proposalId, payload)    // from Agent 3 only
execute(proposalId)                    // callable by anyone (pull-based execution)
```

Output Interface

```
getExecutionStatus(proposalId) -> (queued, executed, failed)
getPayload(proposalId) -> bytes
```

Invariants

- Each proposal executes at most once
- Execution does not depend on external state (payload is self-contained)
- No reentrancy into Layer 2 core logic

Agent 5: Identity Binding Agent

Domain

Bridge between Layer 1 (Sybil Resistance) and Layer 2 (Voting).

Responsibilities

- Verify ZK proofs / DID attestations from Layer 1
- Maintain the canonical 1 Identity = 1 Voting Account mapping
- Invalidate voting rights if Layer 1 flags an identity as compromised
- Gate all voting functions: no unverified identity can interact with Agents 1-4

State Ownership

```
mapping(did => address) votingAccount
mapping(address => bool) isVerified
mapping(address => bool) isCompromised
```

Input Interface

```
registerIdentity(did, zkProof, votingAddress) // from Layer 1
flagCompromised(did)                        // from Layer 1 or Layer 5 veto
revokeIdentity(did)                         // from Layer 1
```

Output Interface

```
isEligible(votingAddress) -> bool
getDid(votingAddress) -> did
```

Invariants

- One DID maps to exactly one voting address at any time
- Compromised identities are immediately ineligible (credits frozen in Agent 1)
- All Layer 2 entry points check isEligible before processing

This agent is the single point of entry for all Layer 2 voting operations. A compromise here breaks the entire Sybil resistance guarantee.

Agent 6: Parameter Registry Agent

Domain

Governance parameter storage and bounded mutation.

Responsibilities

- Store all tunable parameters: quorum thresholds, voting duration, credit regeneration rate, cost bounds
- Validate parameter adjustments proposed by Layer 3 (Intelligence) against hardcoded min/max bounds
- Enforce timelock on parameter changes (from Layer 5)
- Serve as the single source of truth for all other agents

State Ownership

```
uint256 minQuorum, maxQuorum
uint256 minDuration, maxDuration
uint256 regenerationRate
mapping(bytes32 => Parameter) parameters
```

Input Interface

```
proposeParameterChange(paramKey, newValue)    // from Layer 3 only
executeParameterChange(paramKey)              // after timelock expires
emergencyRevert()                             // from Layer 5 veto, resets to safe baseline
```

Output Interface

```
getParameter(paramKey) -> uint256
isWithinBounds(paramKey, value) -> bool
```

Invariants

- No parameter ever exceeds its hardcoded bounds (enforced at the code level)
- Changes sit in timelock before activation
- Emergency revert resets all parameters to deployment defaults

This agent embodies the Safety Principle: automated parameter adjustment is mathematically bounded. Hardcoded bounds prevent runaway feedback loops even if Layer 3 is compromised.

Agentic Coding Specifications

Implementation Rules per Agent

For each agent, an AI coding agent must implement the following rules:

Agent 2 (Quadratic Math Engine)

- Pure functions only
- Fuzz-test cost = votePower^2 across all edge cases
- Priority: CRITICAL -- mathematical correctness

Agent 1 (Voice Credit Ledger)

- Reentrancy guards on all state-changing functions
- Balance invariants checked after every operation
- Priority: CRITICAL -- fund/credit safety

Agent 5 (Identity Binding)

- Signature verification on all identity claims
- No duplicate DIDs allowed at any time
- Priority: CRITICAL -- Sybil resistance

Agent 6 (Parameter Registry)

- Bound checks in constructor and every setter
- Timelock enforcement on all non-emergency changes
- Priority: HIGH -- prevents runaway optimization

Agent 3 (Proposal Lifecycle)

- State machine with strict enum transitions
- Timestamp checks against Agent 6 parameters
- Priority: HIGH -- governance integrity

Agent 4 (Execution Router)

- Nonce-based ordering for all executions
- Event emission for every state change
- No callbacks into Layer 2 core logic
- Priority: HIGH -- execution safety

Recommended Agent Execution Order

Agents should be implemented in dependency order to minimize blocking and maximize parallel testing:

1. Agent 2 (pure math, no dependencies)
2. Agent 6 (parameters, no dependencies)
3. Agent 1 (depends on Agent 6 for rates)
4. Agent 5 (depends on Layer 1 interface spec)
5. Agent 3 (depends on Agents 1, 2, 6)
6. Agent 4 (depends on Agent 3)

This ordering ensures that foundational agents (math, parameters) are hardened before higher-level agents (lifecycle, execution) are built on top of them.

Core Design Principles

1. Single Responsibility

Each agent owns exactly one functional domain. No agent handles two distinct concerns. This minimizes cognitive load per coding agent and simplifies formal verification.

2. Interface Isolation

Agents communicate only through declared interfaces. No agent may directly access another agent's internal state. All cross-agent calls are explicit, logged, and auditable.

3. Determinism Guarantee

The core Layer 2 (Agents 1, 2, 3, 4) must be purely deterministic. No external calls, no oracles, no timestamp-dependent logic in the math engine. This ensures reproducible governance outcomes.

4. Immutable Core

Agent logic is non-upgradable. Parameter changes are tunable (via Agent 6), but the mathematical and structural rules are burned into the deployment. This ensures baseline trust for all participants.

5. Defense in Depth

Safety is enforced at multiple layers: mathematical bounds (Agent 2), state machine guards (Agent 3), parameter bounds (Agent 6), identity verification (Agent 5), and the Layer 5 veto/timelock. No single point of failure can compromise the system.

6. Agentic Testability

Each agent must be independently unit-testable with mocked dependencies. Integration tests verify interface contracts. Fuzz tests verify mathematical invariants. Formal verification targets Agents 1, 2, and 5.

Summary

This horizontal decomposition transforms Layer 2 from a monolithic smart contract into six specialized, independently verifiable agents. The structure directly supports agentic coding workflows:

- Multiple AI agents can implement different agents in parallel
- Each agent has clear interfaces, invariants, and test requirements
- The decomposition preserves the architecture's core principles: Sybil resistance, optimization safety, and deterministic execution
- Integration points are explicit, minimizing the risk of interface mismatches

End of Document